Andromeda Spec 05: Dispatch and Issue

Oct 10, 2026 · @Jerry

Part of the Andromeda spec set. Shared types and conventions are defined in file 00.

Modules: `dispatch`, `issue_queue` · **Milestone:** Instruction dispatch, issue and register renaming

1. What this unit does

Dispatch is where instructions leave program order. It gives each renamed uop a ROB entry, an issue queue entry and, for loads and stores, a load or store queue entry. The issue queues then hold uops until their operands are ready and send them to the execution ports, oldest ready uop first.

There are three issue queues: integer (16 entries, feeding P0 and P1), FP (8 entries, P2) and memory (8 entries, P3). All three are instances of one generic `issue_queue` module.

2. How it works

2.1 Dispatch: the last in-order step

Dispatch must happen in program order because the indices it hands out define each instruction's age:

- `rob_id` is the instruction's position in the ROB, used to report completion and to compare ages everywhere.
- `lq_id` and `sq_id` are positions in the load and store queues. A load's `sq_id` also marks which stores are older than it (file 09).

The group of up to 4 dispatches only if every structure it needs has room: enough ROB entries, enough entries in each issue queue it uses, enough load and store queue entries. Otherwise the whole group waits. Uops go to queues by `fu_type`: ALU, BRU and CSR to the integer queue, FPU to the FP queue, LSU to the memory queue.

2.2 Inside an issue queue

Each entry holds a uop, a valid bit, and a ready bit per source (from rename). Every cycle:

1. **Wakeup.** Each entry compares its not-ready source tags with the 4 wakeup buses. On a match, that source becomes ready.
2. **Select.** Among entries with every source ready, each port picks the oldest by `rob_id`.
3. **Issue.** The selected uop goes to the port's register read stage, and its entry is freed.

A uop waiting on a source doesn't block anything. Younger independent uops are selected past it, which is the whole point of out-of-order execution.

2.3 Example

The integer queue holds three uops. `p40` comes from a load still in flight.

Entry	Uop	rob_id	Sources	Ready?
A	add p41, p40, p4	1	p40 busy, p4 ready	No

Entry	Uop	rob_id	Sources	Ready?
B	addi p42, p5, 1	2	p5 ready	Yes
C	sub p43, p42, p41	3	both busy	No

This cycle, B is the only ready uop and issues even though A is older. When B writes back, the wakeup for p42 sets one of C's sources ready. When the load returns, p40 wakes A. After A writes back, p41 wakes C's other source and C can issue.

2.4 Two ports from the integer queue

The integer queue selects up to two uops per cycle:

- **P1** picks the oldest ready BRU or CSR uop, or if there is none, the oldest ready ALU uop.
- **P0** picks the oldest ready ALU uop not taken by P1.

Branches resolve sooner when they don't have to compete with ALU work, and only P1 can execute them.

2.5 Serializing uops

CSR instructions, FENCE, FENCE.I, MRET, ECALL and EBREAK are marked `serialize`. They are only eligible for select when their `rob_id` equals the ROB head, so they always see fully committed state. The ROB flushes after they commit where needed (file 10).

2.6 Wakeup timing in v1

In v1, wakeup is broadcast in the same cycle as writeback. A uop that depends on a 1-cycle ALU op is therefore selected the cycle after its producer writes back, which leaves a gap between dependent ALU instructions. Early wakeup (broadcasting the tag when the producer issues, so the dependent issues right behind it) is a planned improvement. It only changes *when* units drive `wakeup`, which is why `wakeup_t` is separate from `wb_t`.

3. Interface

3.1 dispatch

Signal	Dir	Width	Description
`flush`	in	1	`be_flush`
`in_valid` / `in_uop` / `in_ready`	in / in / out	4 / 4 × uop_t / 1	From rename
`rob_alloc_valid` / `rob_alloc_uop`	out	4 / 4 × uop_t	New ROB entries
`rob_alloc_id`	in	4 × 6	ROB ids for the 4 slots (tail + i)
`rob_free_cnt`	in	6	Free ROB entries
`iq_int_enq_valid` /	out	4 each	Per-slot enqueue to each queue

Signal	Dir	Width	Description
iq_fp_enq_valid / iq_mem_enq_valid			
iq_enq_uop	out	4 × uop_t	Same uops, rob_id and lq/sq ids filled in
iq_int_free_cnt / iq_fp_free_cnt / iq_mem_free_cnt	in	5 / 4 / 4	Free entries per queue
lq_alloc_valid / sq_alloc_valid	out	4 / 4	Load and store queue allocation
lq_alloc_id / sq_alloc_id	in	4 × 4 / 4 × 4	Allocated indices with wrap bit
sq_tail	in	4	Store queue tail with wrap bit. A load's sq_id = sq_tail + the number of stores ahead of it in the same group; this is its older-store marker
lq_free_cnt / sq_free_cnt	in	4 / 4	Free load and store queue entries

3.2 issue_queue #(DEPTH, NUM_ISSUE)

Signal	Dir	Width	Description
flush	in	1	Empties the queue
enq_valid / enq_uop	in	4 / 4 × uop_t	From dispatch
free_cnt	out	log2(DEPTH)+1	Free entries
wakeup	in	4 × wakeup_t	Tags produced this cycle
rob_head_id	in	6	For serializing uops and age comparison
iss_valid / iss_uop	out	NUM_ISSUE / NUM_ISSUE × uop_t	Selected uop per port
iss_ready	in	NUM_ISSUE	The port's unit can accept a uop this cycle

Instances: integer (DEPTH 16, NUM_ISSUE 2), FP (8, 1), memory (8, 1).

4. Implementation notes

- **Non-collapsing queue.** The simplest design writes new uops into any free entries and never moves entries. Age comes from comparing rob_ids. A collapsing queue, which shifts entries so the oldest is always at the bottom, makes select a simple priority encoder but moves a lot of data every cycle. Start with non-collapsing.

- **Finding free entries.** Dispatch can write up to 4 uops per cycle, so the queue needs "find the first 4 free entries" logic. The shared find-first-N picker (file 00) does this.
- **Oldest-first select.** Comparing every ready entry's `rob_id` against every other is expensive for 16 entries. Two cheaper options: an *age matrix* (one bit per entry pair, set at enqueue, saying "entry i is older than j"), or comparing `rob_id`s relative to the ROB head so plain unsigned comparison works. The age matrix is the standard choice in real cores and is fast.
- **Wakeup cost.** Every entry compares up to 3 source tags with 4 buses each cycle: 192 comparators of 7 bits for the integer queue. Compare only the 6-bit index plus the `is_fp` bit, and skip sources that are already ready.
- **The critical loop.** Wakeup, then select, then the next wakeup is the classic timing loop of an out-of-order core. Keep it inside one cycle in v1; if it misses timing, the fix is usually to simplify select (approximate oldest-first) rather than add a stage.
- **Don't lose wakeups between rename and enqueue.** A uop's ready bits are captured at rename, but it enters the queue a cycle later. A wakeup broadcast during dispatch must still reach it: either dispatch checks the wakeup buses against the uops it holds, or the queue checks them on the enqueue path. Missing this causes uops that wait forever, one of the most common bugs in this kind of design.
- **Port back-pressure.** When `iss_ready` for a port is low (FP divide busy, for example), the queue must not select for that port and must not lose the entry.
- **Dispatch counts are conservative.** It's fine for dispatch to stall if a queue has fewer than 4 free entries even when the group needs fewer, if that simplifies timing. Measure later whether it matters.

5. Verification checklist

- ☐ Unit test of issue_queue against a software model with random enqueue, wakeup and back-pressure
- ☐ Oldest-first select verified, including across `rob_id` wraparound
- ☐ Integer queue: P1 prefers BRU/CSR, P0 never takes a BRU or CSR uop
- ☐ Serializing uop issues only at ROB head
- ☐ Wakeup arriving the same cycle an entry is enqueued is not lost
- ☐ Dispatch stalls whole group when any needed structure is short; no partial dispatch
- ☐ Flush empties all queues and drops in-progress dispatch